

별무리

iOS 온디바이스 AI 개발 / 김민석

스마트폰에서 캐릭터와 대화하고 알람과 일정 같은 기기 기능을 연결하는 iOS 앱입니다. 소형 언어 모델을 기기 안에서 실행하며, 저장한 대화 내용을 검색해 이후 답변에 사용합니다.

기간 2026.05 - 현재
참여 과정 AI-SW 마에스트로 서울 제17기
담당 온디바이스 AI 개발
진행 상태 TestFlight 사용자 테스트 중, 정식 출시 준비

전체 구조와 사용 기술

구성 요소	기술	역할
클라이언트	Unity	사용자 입력과 세션 정보를 전달하고, 생성 중인 답변을 표시합니다.
AI 요청 처리	Swift	대화과 도구 요청을 분기하고, 기억을 검색해 모델 입력을 구성하며, 답변 조각을 반환합니다.
언어 모델 추론	Gemma 4 E2B / LiteRT-LM	기기 CPU에서 대화 답변과 도구 호출 인자를 생성합니다.
임베딩과 분류	EmbeddingGemma / MLP	실행 의도를 분류하고, 도구 선택과 기억 검색에 쓸 문장 벡터를 생성합니다.
장기 기억	SQLite	저장 대상 발화와 벡터를 보관하고, 현재 질문과 관련 있는 기억을 검색합니다.

담당 구현

- Unity 요청을 C API와 Swift 브릿지를 거쳐 LiteRT-LM 모델 호출로 연결했습니다.
- EmbeddingGemma와 SQLite 기반 장기 기억의 저장과 검색을 구현했습니다.
- 실행 의도 분류와 도구 선택, KV 캐시 재사용과 파일 저장 및 복원을 구현했습니다.
- 분류기 평가, 추론 결과 비교, 아이폰 자원 측정을 위한 평가 환경을 개발했습니다.

소스 코드: <https://github.com/mornye-minor-gallery/PetAI-AI>

기술 문서: <https://pysunn.me/docs-beolmuri-ai/>

일반 대화를 도구 실행 요청으로 오인하는 문제

문제

- 여러 도구 설명을 한 번에 넣은 프롬프트에서 소형 모델이 일반 대화도 실행 요청으로 판단했습니다.
- 임베딩 유사도로 바꾼 뒤에도 일반 대화 24건 중 11건이 호출 후보로 분류됐습니다.

원인과 분류 방식

- 기능 문의, 과거 경험, 인용문도 도구 설명과 유사도가 높아 실행 의도와 도구 종류를 분리해 판단하도록 구성했습니다.
- EmbeddingGemma의 768차원 벡터를 입력받는 경량 MLP를 도구 선택 앞에 추가했습니다.

학습 데이터 수정

- 초기 MLP의 일반 대화 오인을 줄이려고 비실행 사례를 보강했지만, 실제 실행 요청 탐지는 163건에서 118건으로 감소했습니다.
- 같은 기능의 실행 요청과 비실행 문장을 함께 학습하도록 바꾸자, 실제 실행 요청 탐지가 160건으로 회복됐습니다.

학습 구성별 호출 후보 판정

실제 실행 요청 168건과 일반 대화 24건으로 비교했습니다.

학습 구성	일반 대화 오인	실제 요청 탐지
초기 MLP	16/24	163/168
비실행 사례만 보강	2/24	118/168
기능별 실행/비실행 쌍 보강	2/24	160/168
위 구성에 한국어 비실행 사례 추가	1/24	159/168

적용

- MLP를 통과한 발화만 도구별 대표 벡터와 비교하고, 선택한 도구 하나의 정의를 모델에 전달해 JSON 인자를 생성했습니다.
- 한 요청에서 도구 하나만 선택하고, 여러 도구가 기준을 넘으면 요청을 나누도록 안내했습니다.

반복 대화의 대기 시간을 줄인 KV 캐시 재사용

사용자 피드백과 수정

- TestFlight 사용자에게 첫 답변이 늦게 나온다는 피드백을 받아 응답 대기 시간을 개선했습니다.
- 매 요청에서 같은 고정 프롬프트를 다시 처리하는 비용을 줄이려고 CachedSession으로 메모리의 KV 캐시를 다음 요청에 재사용했습니다.

메모리 KV 캐시 재사용 측정

iPhone 17에서 같은 5턴 대화를 조건별로 3회 실행해 총 30개 요청을 비교했습니다.

첫 출력 대기 시간	요청마다 새 세션	캐시 재사용
전체 평균	4.920초	1.274초
후속 요청 평균	4.872초	0.446초

첫 출력 대기 시간은 전체 평균 74.1%, 후속 요청 평균 90.8% 줄었습니다. 답변 15쌍이 일치했습니다.

앱 재실행 후 재사용을 위한 파일 저장과 복원

남은 문제와 엔진 수정

- 앱을 종료하면 메모리의 캐시가 사라져, 재실행 시 같은 고정 프롬프트를 다시 계산해야 했습니다.
- 사용하던 LiteRT-LM의 공개 C API에 상태 저장과 복원 기능이 없어 엔진을 포크하고 CPU 세션 상태 전송 API를 추가했습니다.
- 엔진에서 KV 텐서, 처리 위치, 토큰 ID, 샘플러 상태를 전송하고, Swift에서 파일 저장과 복원, 모델 정보와 체크섬 검사를 담당했습니다.